Two-Coloured Trees, Container Composition, and Compiler Intermediate Representations

Robin Langer
For discussion with Neil Ghani

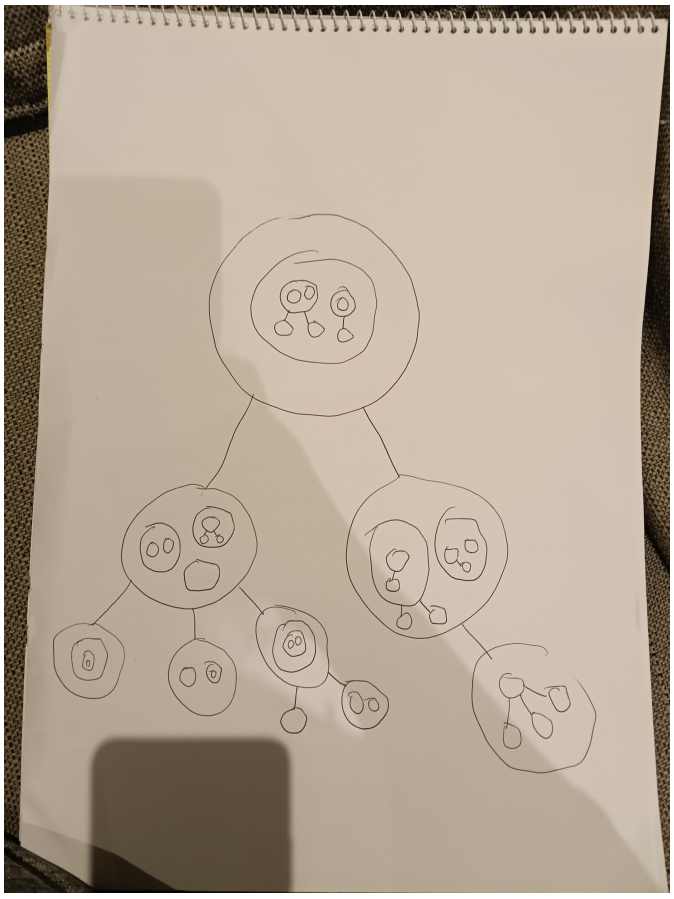
July 2026

Abstract

We observe that the composition of containers (Abbott–Altenkirch–Ghani) produces trees with two structurally distinct kinds of edge: *outer* edges connecting nodes in the composite tree, and *inner* edges encoding the structure within each node. When the composition is iterated to arbitrary depth, we obtain the free monad on the container functor. We apply this perspective to compiler intermediate representations (IRs), where the two colours correspond to *term structure* (how subterms compose) and *binding structure* (how variables come into scope). Compilation passes become container morphisms, and administrative normal form (ANF) conversion is monad multiplication—collapsing one colour into the other.

1 The Diagram

The following diagram shows a tree with two colours of edge. Outer edges (solid lines) connect nodes. Inner structure (circles nested within circles) encodes the shape at each node—its positions and internal wiring. Each innermost circle is a *port*: a position where an outer edge attaches to a child.



The diagram has two kinds of structural relationship:

1. **Outer edges** (solid lines between circles): the tree structure connecting nodes to children. These are the positions of the *outer* container.
2. **Inner structure** (circles nested within circles, connected by thin lines): the internal wiring of each node. The leaves (ports) of the inner tree are the positions where outer edges attach.

2 Container Composition Produces Two Colours

Recall that a container $S \triangleleft P$ is a set S of shapes with a family of positions $P : S \to \mathbf{Set}$, and its extension is

$$[\![S \triangleleft P]\!](X) = \coprod_{s \in S} X^{P(s)}.$$

Definition 2.1 (Composition of containers)**.** Given containers $C_1 = S_1 \triangleleft P_1$ and $C_2 = S_2 \triangleleft P_2$, their composition $C_1 \circ C_2$ is the container with:

- Shapes: $\coprod_{s_1 \in S_1} S_2^{P_1(s_1)}$ (an outer shape s_1 plus a choice of inner shape at each outer position),
- Positions at shape (s_1, v): $\coprod_{p \in P_1(s_1)} P_2(v(p))$ (an outer position p plus an inner position at $v(p)$).

Its extension satisfies $[\![C_1 \circ C_2]\!] \cong [\![C_1]\!] \circ [\![C_2]\!]$.

The shape of a composite element is a **tree of depth 2**: an outer shape s_1 whose positions are filled with inner shapes $v(p) \in S_2$. The two colours in the diagram are exactly the two levels of this nesting:

- Colour 1 (outer edges) corresponds to the outer positions $p \in P_1(s_1)$.
- Colour 2 (inner structure) corresponds to the inner positions $q \in P_2(v(p))$.

Remark 2.2. In the diagram, the self-similar structure—inner nodes themselves containing structure—suggests more than two levels. This is the iterated composition C^n, or in the limit, the *free monad*.

3 The Free Monad: Arbitrary Depth

If C is a container, its free monad $\mathrm{Free}(C)$ is the container whose elements are *finite trees of arbitrary depth* over C:

$$\mathrm{Free}(C)(X) = X + C(\mathrm{Free}(C)(X))$$

or equivalently, as a W-type (well-founded tree type):

$$\mathrm{Free}(C)(X) = \mu T.\, X + \coprod_{s \in S} T^{P(s)}.$$

An element is either a leaf labelled by $x \in X$, or a node with shape s and a child at each position. The number of "colours" is the depth of the tree: a path from root to leaf passes through n node boundaries (each a colour-transition), where n varies across branches.

The monad structure is:

- **Unit** $\eta : X \to \mathrm{Free}(C)(X)$: embed x as a leaf.
- **Multiplication** $\mu : \mathrm{Free}(C)(\mathrm{Free}(C)(X)) \to \mathrm{Free}(C)(X)$: given a tree whose leaves are themselves trees, *graft* the leaf-trees onto the outer tree. This **dissolves one level of nesting**—it merges two adjacent colours into one.

The diagram on the first page, with its self-similar nesting, is an element of $\mathrm{Free}(C)(X)$ for some C—a tree of trees of trees, with μ unapplied.

4 The Metacircular Evaluator

The purest example of a two-coloured tree—and the monad it forms—is a Lisp interpreter written in Lisp.

4.1 S-Expressions as a Container

The container for Lisp S-expressions is remarkably simple:

$$S \triangleleft P \quad \text{where} \quad S = \{\mathsf{atom},\ \mathsf{cons}\}, \qquad P(\mathsf{atom}) = \emptyset, \qquad P(\mathsf{cons}) = \{0, 1\}.$$

An atom has no positions (it is a leaf). A cons cell has two positions: car and cdr. The extension is

$$[\![S \triangleleft P]\!](X) = X^0 + X^2 = 1 + X \times X.$$

The free monad on this container is the type of S-expressions:

$$\mathsf{SExpr}(X) = \mathrm{Free}(S \triangleleft P)(X) = \mu T.\, X + 1 + T \times T.$$

An S-expression is either a variable ($x \in X$), an atom, or a cons of two S-expressions. This is the recursive type that every Lisp implementation defines.

4.2 Quoting is the Unit, Eval is Multiplication

A Lisp interpreter written in Lisp operates on two levels of S-expression:

- **Colour 1**: the interpreter's own code—the S-expressions that *are* the interpreter.
- **Colour 2**: the program being interpreted—S-expressions that are *data* to the interpreter, received as quoted input.

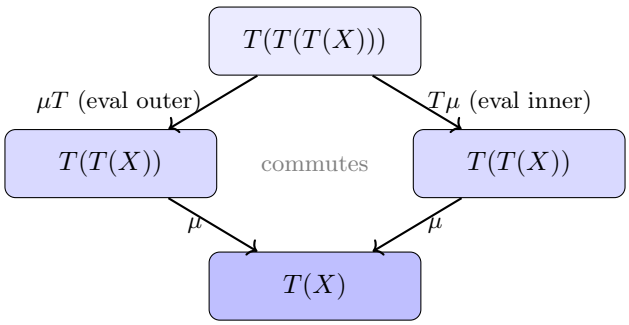
An expression like `(eval '(+ 1 2))` is an element of $T(T(X))$: the outer `eval` call is colour 1, and the quoted `(+ 1 2)` inside it is colour 2.

The monad operations are:

- **Unit η = quoting.** The quote operator `'` wraps a value in one extra layer of tree structure: it takes an S-expression in $T(X)$ and lifts it to $T(T(X))$ by adding a colour boundary. The value `(+ 1 2)` becomes the datum `'(+ 1 2)`—the same tree, but now enclosed in a second colour.
- **Multiplication μ = eval.** `eval` takes a tree-of-trees (a program representing a program) and collapses it to a single tree (the result of running it). This dissolves the boundary between colour 1 and colour 2.

The monad laws are the correctness conditions for the metacircular evaluator:

1. $\mu \circ \eta = \mathrm{id}$: evaluating a quoted expression returns the original value. `(eval '(+ 1 2))` $= 3$.
2. $\mu \circ T\eta = \mathrm{id}$: quoting inside an expression and then evaluating the whole thing also returns the original.
3. $\mu \circ \mu T = \mu \circ T\mu$ (associativity): for three levels of nesting—a Lisp interpreter interpreting a Lisp interpreter interpreting a program—it does not matter which pair of adjacent colours you collapse first. The result is the same.



4.3 Homoiconicity is the Monad Condition

Most languages have $T_1(T_2(X))$: the interpreter is in language T_1, the interpreted program is in language T_2. This is composition of *different* containers. To form a monad you need $\mu : T \circ T \to T$—both layers must be the *same* T.

Homoiconicity ("code is data") is exactly the condition that makes this possible. In a homoiconic language, the container for the language's syntax is the same container used to represent data. The metacircular evaluator exists *because* $T = T$, giving a monad rather than merely a composition of distinct functors.

Remark 4.1. Non-homoiconic languages can still have interpreters, but they are container *morphisms* $(u, f^\sharp) : C_{\text{source}} \to C_{\text{target}}$ rather than monad multiplications. The difference: a morphism maps between two containers; multiplication maps a container to itself. Only the latter gives you the tower property—interpreting an interpreter interpreting an interpreter, with associativity guaranteeing confluence.

5 Snoc-Trees and Composition

The two-coloured tree has a natural directional asymmetry: colour-1 edges branch (a node may have many children), while colour-2 edges are sequential (each binding extends the context by one variable). The colour-2 spine is a *snoc-list*.

5.1 Snoc-Lists as Containers

A **snoc-list** builds rightward by appending:

$$\mathsf{Lin} :> a_1 :> a_2 :> a_3 \qquad \text{(three elements, built right-to-left)}$$

As a container, the snoc-list functor L has:

- Shapes $S = \mathbb{N}$ (the length).
- Positions at shape n: $P(n) = \{0, 1, \ldots, n-1\}$ (slots indexed from the right, matching de Bruijn convention: position 0 is the most recently appended element).

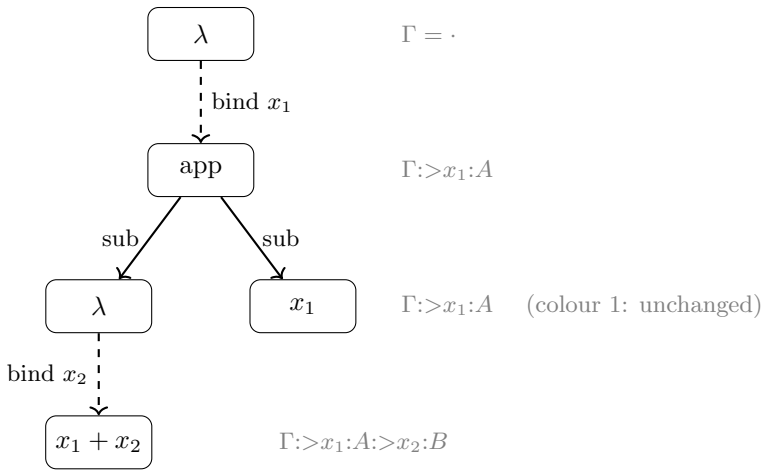
Its extension is $[\![L]\!](X) = \coprod_{n \geq 0} X^n$—finite sequences, the same as the list container but with the opposite construction bias.

5.2 Contexts are Snoc-Lists

In dependent type theory, a typing context is a snoc-list of typed variable declarations:

$$\Gamma = (\cdot :> (x_1 : A_1) :> (x_2 : A_2(x_1)) :> (x_3 : A_3(x_1, x_2))).$$

Each $:>$ is a colour-2 edge in the two-coloured tree. A path from the root of the AST to a node, reading only the binding edges (colour 2) encountered along the way, yields the snoc-list context at that node. Colour-1 edges do not change the context; colour-2 edges snoc onto it.



The snoc-list at each node is the *accumulated* colour-2 path. De Bruijn index 0 refers to the most recent $:>$ (the last binding edge crossed). This is why snoc-lists are the natural representation: access to the most recent element is $O(1)$, matching the frequency with which bound variables are used near their binders.

5.3 Composition of Snoc-Trees

A **snoc-tree** is a tree built by iterated snoc—equivalently, it is the free monad on the snoc-list container. But the interesting composition is not snoc-list with snoc-list; it is *snoc-list composed with a branching container*. This is exactly the two-coloured tree.

Let $B = S_B \triangleleft P_B$ be a branching container (the syntax: shapes are constructors, positions are argument slots) and let $L = \mathbb{N} \triangleleft [n]$ be the snoc-list container. The composition $B \circ L$ has:

- Shapes: a syntax constructor $s \in S_B$ together with, at each position $p \in P_B(s)$, a natural number n_p (the length of the snoc-list—how many bindings this edge introduces).
- Positions at shape $(s, \{n_p\})$: $\coprod_{p \in P_B(s)} [n_p]$ (an argument slot p plus a de Bruijn index within the n_p new bindings at that slot).

An element of $[\![B \circ L]\!](X)$ is a syntax node where each argument slot carries a *telescope* of new bindings before reaching the child term. This is precisely the standard notion of a syntax constructor in dependently typed languages: each argument may bind zero or more variables before the subterm.

Example 5.1 (The Π-type constructor)**.** The shape is Π with two positions: the domain and the codomain. The domain binds nothing ($n_1 = 0$); the codomain binds one variable ($n_2 = 1$). So the composite shape is $(\Pi, [0, 1])$, and the positions are $\{(\mathrm{dom}, \varnothing)\} \cup \{(\mathrm{cod}, 0)\}$—the domain has no new de Bruijn indices, the codomain introduces index 0.

Example 5.2 (Telescopic binders)**.** A dependent record type with fields $\{(x_1 : A_1),\ (x_2 : A_2(x_1)),\ (x_3 : A_3(x_1, x_2))\}$ has a single position with $n = 3$. The three bindings form a snoc-list $\cdot :> x_1 :> x_2 :> x_3$, and the positions within this telescope are de Bruijn indices $\{0, 1, 2\}$. The whole structure is one element of $B \circ L$.

The composition $L \circ B$ goes the other way: a snoc-list whose elements are trees. This is a **sequence of compilation passes**—each element of the snoc-list is an IR tree, appended in order. The most recent pass (index 0) is the current IR; earlier passes are history. The snoc structure gives $O(1)$ access to the current state while preserving the full provenance chain.

The monad multiplication. For $B \circ L$, the multiplication $\mu : (B \circ L) \circ (B \circ L) \to B \circ L$ concatenates telescopes. If a binding introduces a telescope Δ_1, and within that scope another binding introduces Δ_2, flattening merges them:

$$\Delta_1 :> \Delta_2 \quad \longrightarrow \quad \Delta_1 +\!\!+ \Delta_2.$$

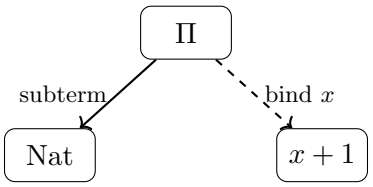
This is the `let`-flattening operation of ANF, generalised from single bindings to telescopes: nested scopes are concatenated into a single snoc-list of bindings.

6 Application to Compiler Theory

Compiler intermediate representations are trees. The container perspective makes the *two-colour structure* of these trees explicit, and gives precise meaning to compilation passes and normal forms.

6.1 Syntax Trees Have Two Colours

The abstract syntax tree (AST) of a dependently typed language has two structurally distinct kinds of descent:



⟶ Colour 1: subterm edge (context unchanged)
⇢ Colour 2: binding edge (context extended with x)

- **Colour 1** (subterm edges): descend into a subterm without changing the variable context. In $f(t)$, both f and t live in the same context as the application node.
- **Colour 2** (binding edges): descend under a binder, extending the context. In $\lambda x.\, body$, the edge to *body* adds x to the context.

In container terms: the AST is a container whose shapes are syntax constructors (Π, λ, app, let, ...) and whose positions are argument slots. The positions decompose into *non-binding* positions (colour 1) and *binding* positions (colour 2). A binding position p carries a variable into scope below it; the context at that child is $\Gamma, x : A$ rather than Γ.

This decomposition is precisely the data of a **second-order container** in the sense of Fiore, Plotkin, and Turi: an operation with some first-order inputs (colour 1) and some higher-order inputs (colour 2).

6.2 Compilation Passes as Container Morphisms

A compilation pass from IR_1 to IR_2 is a container morphism $(u, f^\sharp) : S_1 \triangleleft P_1 \to S_2 \triangleleft P_2$:

- **Forward on shapes** $u : S_1 \to S_2$: translate each syntax node. A `let`-expression becomes a `move` instruction; a function application becomes a `call`.
- **Backward on positions** $f^\sharp_s : P_2(u(s)) \to P_1(s)$: for each position in the output node, say which position of the input node supplies its value. This is **provenance**: every subterm in the output traces back to a subterm in the input.

The backward map is not optional decoration. It is half of the container morphism. A compiler pass without provenance is not a container morphism—it is a partial function on shapes that discards the position structure. The representation theorem (Abbott–Altenkirch–Ghani 2005) says that *every* natural transformation between container extensions arises from a unique container morphism. So if your pass is natural (it does not inspect the data at positions, only the shapes), it automatically has a backward map.

Remark 6.1. This connects to MacBeth's observation that directed-container morphisms are *cofunctors*, not functors. A compilation pass goes forward on syntax (shapes) and backward on provenance (positions)—the same variance as a cofunctor, and the same variance as the Put-half of a lens.

6.3 ANF Conversion is Monad Multiplication

Administrative normal form (ANF) eliminates nested intermediate expressions by naming every subcomputation:

$$\underbrace{\texttt{let}\ x = (\texttt{let}\ y = e_1\ \texttt{in}\ e_2)\ \texttt{in}\ e_3}_{\text{nested: tree of trees}} \quad\longrightarrow\quad \underbrace{\texttt{let}\ y = e_1\ \texttt{in let}\ x = e_2\ \texttt{in}\ e_3}_{\text{flattened: single tree}}$$

The left side is an element of $T(T(X))$—a tree whose leaves are themselves trees (two colours of nesting). The right side is an element of $T(X)$—a single tree (one colour). The transformation is exactly monad multiplication:

$$\mu : T^2(X) \longrightarrow T(X).$$

ANF conversion collapses one level of nesting, merging colour 2 (the inner tree of the nested `let`) into colour 1 (the outer tree).

More generally, any **flattening pass** in a compiler—ANF, CPS, A-normal form, monadic normal form—is μ for some monad on the syntax container. The free monad $\mathrm{Free}(C)$ gives the "fully un-flattened" IR: arbitrary nesting, all colours visible. Applying μ repeatedly drives the representation toward a normal form.

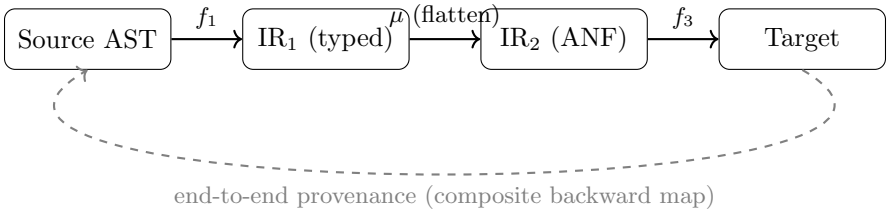
6.4 Multi-Pass Compilation as the Free Monad

A compiler with n passes through distinct IRs is a chain of container morphisms:

$$C_{\text{source}} \xrightarrow{f_1} C_{\text{IR}_1} \xrightarrow{f_2} C_{\text{IR}_2} \xrightarrow{f_3} \cdots \xrightarrow{f_n} C_{\text{target}}$$

Each morphism has a forward map (translation) and a backward map (provenance). The composite $f_n \circ \cdots \circ f_1$ gives **end-to-end provenance**: every position in the target traces back through every intermediate stage to the source.

When the number of passes is not fixed in advance—when the compiler may insert or remove optimisation passes—the appropriate structure is the free monad $\text{Free}(C)$. An element of $\text{Free}(C)$ is a tree of arbitrary depth, each level corresponding to one pass that has not yet been applied. The monad multiplication μ applies a pass (collapses one level); the unit η adds a trivial pass (identity translation).



6.5 Scope Management as Directed Container

A typing context $\Gamma = (x_1 : A_1,\ x_2 : A_2(x_1),\ x_3 : A_3(x_1, x_2))$ is a snoc-list, and the category of contexts and substitutions is a directed container:

- **Shapes**: contexts Γ (lists of typed variable declarations).
- **Positions at** Γ: well-typed terms in context Γ (or, more simply, de Bruijn indices into Γ).
- **Root** $o(\Gamma)$: the identity substitution.
- **Shift** $\oplus$: substitution composition.
- **Downarrow** $\Gamma \downarrow p$: the context reached by applying substitution p.

The comonad structure gives weakening and substitution canonically. The two colours reappear: colour 1 edges (term structure) preserve the context; colour 2 edges (binding) shift the context via $\downarrow$. The scope checker is the functor from the "unscoped" container (raw syntax, all positions equivalent) to the "scoped" directed container (positions carry de Bruijn indices, binding edges shift them).

7 Summary

Container theory	**Compiler theory**	**Colour**
Outer position $p \in P_1(s_1)$	Subterm slot	1 (line)
Inner position $q \in P_2(v(p))$	Binding slot	2 (containment)
Container morphism $(u, f^\sharp)$	Compilation pass	—
Forward u	Syntax translation	shapes
Backward $f^\sharp$	Provenance / blame	positions
Snoc-list container L	Context / telescope	2 (binding spine)
$B \circ L$ (branching $\circ$ snoc)	Syntax with telescopic binders	both
$L \circ B$ (snoc $\circ$ branching)	Sequence of IR passes	both
μ for $B \circ L$	Telescope concatenation (ANF)	merge $2 \to 1$
Free monad Free(C)	Multi-pass IR (unfixed depth)	all
Directed container	Scoped syntax + substitution	—

The diagram on the first page—a tree with two colours of edge, one drawn as lines and the other as containment—is the picture of container composition. When the nesting is self-similar and unbounded, it is the free monad. In compiler theory, the two colours are always present: one for term structure, one for binding. Seeing them as container composition makes the structure of compilation passes, normal forms, and provenance into consequences of the functorial machinery rather than ad hoc constructions.